

TreydView Unterhalt

Wie die App am Laufen gehalten, aktualisiert und im Notfall repariert wird

BUILD m83

Ein praktisches Handbuch für den laufenden Betrieb – geschrieben so, dass es auch ohne tiefe Programmierkenntnisse Schritt für Schritt befolgtbar ist. Jeder Befehl ist zum Abtippen gedacht; jede Fehlermeldung hat eine zugeordnete Lösung.

Projekt: TreydView — Vanilla-JS Trading-Chart-Cockpit

Frontend: panta-rey.github.io/Treydview/ (GitHub Pages)

Backend: pantarey.rey-gafner.workers.dev (Cloudflare Worker)

Chart-Bibliothek: KLineCharts 9.8.12 (lokal gehostet)

Stand: Build m83 (Basis m82)

00 Inhalt & Schnellstart

Wenn du nur etwas ausliefern willst und alles funktioniert, brauchst du eigentlich nur zwei Dinge: die Build-Nummer erhöhen (Kapitel 5) und pushen (Kapitel 6). Der Rest ist für den Fall, dass etwas klemmt.

- 1 **Woraus TreydView besteht** — das mentale Modell
- 2 **Die Dateien** — was wo liegt und wofür es da ist
- 3 **Vier eiserne Regeln** — was man NIE tun darf
- 4 **Das Terminal** — wo alle Befehle hingehen (fang hier an, wenn du noch nie eines benutzt hast)
- 5 **Build-Nummer erhöhen** — damit der Browser die neue Version lädt
- 6 **Ausliefern (Deploy)** — Schritt für Schritt, auch für den Notfall
- 7 **Die Pflichtprüfungen** — vier Checks vor jedem Push
- 8 **Der Cloudflare-Worker** — das Daten-Backend warten
- 9 **Datenquellen** — welche zuverlässig sind, welche zicken
- 10 **Statische Historie (Snapshots)** — die Altdaten-Dateien
- 11 **Troubleshooting** — Symptom → Ursache → Behebung
- 12 **Schnell-Checkliste** — zum Abhaken vor jeder Auslieferung

01 Woraus TreydView besteht

TreydView ist **reines Vanilla-JavaScript**. Das bedeutet: Es gibt **kein Build-System**, keinen Bundler, kein Framework, kein `npm run build`. Was in den Dateien steht, läuft 1:1 im Browser. Das ist die wichtigste Eigenschaft für den Unterhalt: **Datei ändern → hochladen → fertig**. Kein Kompilieren dazwischen. Die App hat zwei Hälften, die man auseinanderhalten muss:

Teil	Wo er liegt	Was er tut
Frontend die App selbst	GitHub Pages <code>panta-rey.github.io/Treydview/</code>	Die eigentliche Oberfläche: HTML, CSS und JavaScript. GitHub liefert diese Dateien gratis als Webseite aus. Änderungen kommen per <code>git push</code> hierher.
Backend der Daten-Helfer	Cloudflare Worker <code>pantarey.rey-gafner.workers.dev</code>	Ein kleines Server-Programm, das Daten holt, die der Browser aus Sicherheitsgründen nicht direkt darf (Gold, Silber, Aktienindizes, Dominanz, M2, Bitstamp-Historie) und sie zwischenspeichert (cacht).

Kurz erklärt: Was ist ein «Worker»?

Ein Cloudflare Worker ist ein winziges Programm, das bei Cloudflare läuft und auf Anfragen antwortet. TreydView nutzt ihn als Zwischenhändler: Der Browser fragt den Worker, der Worker fragt die Datenquelle (z. B. die LBMA für Gold), merkt sich die Antwort für eine Weile (Cache) und gibt sie zurück. Das ist nötig, weil viele Quellen den direkten Zugriff aus dem Browser blockieren (Stichwort CORS).

Chart-Bibliothek: Die Kurscharts zeichnet **KLineCharts 9.8.12**, lokal gehostet unter `js/lib/klinecharts.min.js` und leicht angepasst (die Zoom-Grenze wurde von `1` auf `0.2` gesetzt). Bewusst kein CDN – die App soll nicht ausfallen, wenn ein fremder Server ausfällt.

02 Die Dateien – was wo liegt

So sieht der Projektordner aus. Die Einrückung zeigt Unterordner.

```
Treydview/
├─ index.html           # Gerüst der Seite: alle sichtbaren Elemente + Ladeliste der Skripte
├─ manifest.webmanifest # macht die App auf dem Handy «installierbar» (Home-Screen)
├─ worker-komplett.js   # der GANZE Cloudflare-Worker-Code (Backend)
├─ snapshot.sh         # erzeugt die statischen Historie-Dateien in data/
├─ HANDOFF.md          # NUR LOKAL – wird NIE committet (steht in .gitignore)
├─ README.md / LICENSE / .nojekyll / .gitignore
├─
├─ css/
│  └─ style.css         # das gesamte Aussehen; enthält --tv-build (Build-Tag)
├─
├─ js/
│  ├─ config.js         # Worker-URL, Assets, Zeitrahmen, Indikator-Registry
│  ├─ indicators.js     # Berechnung der 15 Indikatoren
│  ├─ overlays.js       # Zeichenwerkzeuge (Linien, Fib, FRVP ...)
│  ├─ data.js           # Kursdaten laden, Live-Stream, Nachladen beim Scrollen
│  ├─ settings.js       # Einstellungs-Panels der Indikatoren
│  ├─ derivatives.js    # Funding, Open Interest, Long/Short-Ratio
│  ├─ gridbot.js        # Grid-Bot-Rechner
│  ├─ patterns.js       # Chartmuster-Scanner
│  ├─ smc.js            # Smart Money Concepts (FVG / Order Blocks)
│  ├─ ewt.js            # Elliott-Wellen-Scanner
│  ├─ app.js            # Hält alles zusammen; enthält TV_BUILD. Lädt IMMER zuletzt.
│  └─ lib/klinecharts.min.js # die Chart-Bibliothek (lokal, gepatcht)
├─
├─ data/
│  ├─ btcusd-bitstamp.json # BTC/USD Historie ab 2011
│  ├─ ethusd-bitstamp.json # ETH/USD Historie
│  └─ gold-lbma.json       # Gold XAU/USD ab 1968
```

Die Ladeordnung in index.html ist kein Zufall

Ganz unten in `index.html` stehen die Skripte in genau dieser Reihenfolge: `config` → `indicators` → `overlays` → `data` → `settings` → `derivatives` → `gridbot` → `patterns` → `smc` → `ewt` → `app`. **app.js kommt immer zuletzt**, weil es die Bausteine der anderen Dateien zusammensetzt. Diese Reihenfolge nicht durcheinanderbringen.

03 Vier eiserne Regeln

Diese vier Punkte haben in der Vergangenheit am meisten Ärger verursacht, wenn man sie verletzt hat. Sie gelten ausnahmslos.

Regel 1 — Die Worker-URL nie überschreiben

In `js/config.js` steht: `WORKER_BASE_URL: "https://pantarey.rey-gafner.workers.dev"`. Wird diese Zeile beim Austausch einer Datei versehentlich durch einen Platzhalter ersetzt, lädt **keine einzige Sonderdatenquelle mehr** (Gold, Silber, Indizes, Dominanz, M2, Bitstamp). Nach jedem Ersetzen von `config.js` diese Zeile prüfen.

Regel 2 — Der Desktop ist unantastbar

Alle Handy-Anpassungen laufen über `class="mobile-only"`, CSS in `@media`-Blöcken und JavaScript hinter `matchMedia`. Bestehende HTML-Elemente werden nie geändert – es kommen nur neue Elemente mit `class="mobile-only"` hinzu. So kann eine Handy-Änderung den Desktop nie kaputt machen.

Regel 3 — HANDOFF.md nie committen

`HANDOFF.md` ist die lokale Werkstatt-Notiz (was zuletzt gemacht wurde, offene Punkte, Lehren aus Fehlern). Sie steht in `.gitignore` und gehört **nicht** ins veröffentlichte Repo. Vor dem `git add -A` prüfen, dass sie nicht mitgeht.

Regel 4 — Build-Nummer bei JEDER Auslieferung erhöhen

Sonst zeigt der Browser die alte, zwischengespeicherte Version. Wie das geht, steht in Kapitel 5.

04 Das Terminal - wo alle Befehle hingehen

Alle Befehle in dieser Anleitung (die mit `git`, `bash` oder `node` beginnen) werden nicht irgendwo angeklickt, sondern in ein **Terminal** getippt – ein schlichtes Fenster, in das man Text-Befehle schreibt. Wer das noch nie gemacht hat, liest dieses Kapitel einmal in Ruhe; danach sind die folgenden Kapitel selbsterklärend.

Das Terminal öffnen

System	Was du öffnest	Wie
Mac	App «Terminal»	Cmd + Leertaste drücken, Terminal tippen, Enter .
Windows	Git Bash (kommt mit «Git für Windows»)	Im Datei-Explorer in den Projektordner gehen, ins leere Feld rechtsklicken → «Git Bash Here» . Damit ist man gleich im richtigen Ordner.

Warum auf Windows ausgerechnet Git Bash und nicht die normale Eingabeaufforderung? Weil `bash_snapshot.sh` (Kapitel 10) eine Bash-Umgebung braucht, und weil dort alle hier gezeigten Befehle unverändert laufen.

Wie man einen Befehl ausführt

- 1. Eine Zeile eintippen oder hineinkopieren.
 - 2. Enter drücken.
 - 3. Warten, bis der Cursor in einer neuen, leeren Zeile zurückkommt – erst dann ist der Befehl fertig.
- Text, der hinter einem # steht, ist ein **Kommentar/Erklärung** – den nicht mittippen. Einfügen ins Terminal: auf dem Mac Cmd + V, in Git Bash Shift + Einfg oder Rechtsklick.

In den Projektordner wechseln

Fast jeder Befehl muss **im Projektordner** laufen (dort, wo index.html liegt). Hinwechseln mit cd («change directory», Ordner wechseln). Der bequeme Trick, um den Pfad nicht abtippen zu müssen:

```
# cd und ein Leerzeichen tippen, dann den Projektordner ins
# Terminal-Fenster ZIEHEN - der Pfad erscheint von selbst - dann Enter:
cd
```

Ob man am richtigen Ort ist, prüft man mit zwei kleinen Befehlen:

```
pwd # zeigt, in welchem Ordner man gerade ist
ls # listet die Dateien darin - index.html muss dabei sein
```

Auf Windows entfällt das oft
Wenn du das Terminal über **«Git Bash Here»** im Projektordner geöffnet hast, bist du bereits am richtigen Ort – das cd brauchst du dann gar nicht.

Woran man Erfolg oder Fehler erkennt

Kommt der Cursor **ohne** rote Meldung zurück, hat es geklappt. Steht dort ein Wort wie error, fatal oder command not found, ist etwas schiefgelaufen – die Meldung sagt meist selbst, was fehlt. Im Zweifel den Text der Meldung kopieren und nachschlagen.

Einmalig pro Rechner - die Voraussetzungen	
Das Folgende muss man nur ein einziges Mal auf einem neuen Computer erledigen:	
Prüfen mit	Falls es fehlt
git --version zeigt eine Versionsnummer	Git installieren von git-scm.com (auf Windows bringt das gleich Git Bash mit).
Liegt das Projekt schon lokal? Ordner mit index.html vorhanden?	Einmalig herunterladen («klonen»): git clone https://github.com/panta-rey/Treydview.git
python --version für snapshot.sh (Kapitel 10)	Python 3 von python.org (Haken «Add python.exe to PATH»). Windows-Falle: python3 trifft dort oft nur den Microsoft-Store-Platzhalter – teste darum mit python --version. Das Skript snapshot.sh findet den richtigen Namen inzwischen selbst.
node --version nur für die Prüfungen nötig	Node.js installieren von nodejs.org (erst ab Kapitel 7 gebraucht).

05 Build-Nummer erhöhen (Cache-Bruch)

Browser merken sich Dateien, um schneller zu sein. Damit sie nach einer Änderung wirklich die **neue** Datei holen, trägt jede Auslieferung eine neue Build-Nummer. Diese Nummer steht an **drei Stellen**, und alle drei müssen zusammenpassen:

Datei	Was geändert wird	Beispiel
js/app.js	die Konstante TV_BUILD	const TV_BUILD = "m84";
css/style.css	die Variable --tv-build	:root { --tv-build: "m84"; }
index.html	alle 14 Vorkommen von ?v=...	...style.css?v=m84

Die 14 Stellen in index.html sind: das Manifest, die CSS-Datei, die Chart-Bibliothek und die 11 JS-Skripte. Am schnellsten alle auf einmal mit einem Suchen-Ersetzen. In der **Git-Bash** (kommt mit «Git für Windows») so:

```
# im Wurzelordner des Projekts ausführen; ALT/NEU anpassen
OLD=m83; NEW=m84
sed -i "s/?v=$OLD/?v=$NEW/g" index.html
sed -i "s/const TV_BUILD = \"$OLD\"/const TV_BUILD = \"$NEW\"/" js/app.js
sed -i "s/--tv-build: \"$OLD\"/--tv-build: \"$NEW\"/" css/style.css
```

Ohne Git-Bash geht es genauso von Hand: in jedem der drei Dateien mit **Suchen & Ersetzen** (im Editor) m83 → m84 ersetzen. In index.html «Alle ersetzen», in app.js und style.css nur die eine Zeile.

Die eingebaute Stolperdraht-Warnung

Beim Start vergleicht die App selbst `TV_BUILD` (aus JS) mit `--tv-build` (aus CSS). Stimmen sie überein, steht in der Browser-Konsole (F12) grün: `[TreydView] Build m84 – CSS und JS aktuell.` Stimmen sie nicht überein, kommt `[TreydView] VERSIONSKONFLIKT ...`. Das ist der beste Test, ob man eine Stelle vergessen hat.

06 Ausliefern (Deploy auf GitHub Pages)

«Deploy» heisst: die neue Version ins Netz stellen. Bei TreydView geschieht das nicht über einen Knopf, sondern indem man die Dateien zu GitHub **hochlädt** – die Website `panta-rey.github.io/Treydview` aktualisiert sich danach von selbst. Alle folgenden Zeilen werden im **Terminal** eingegeben (siehe Kapitel 4), und zwar im Projektordner.

Standard-Ablauf (saubere Historie)

```
# in den Projektordner wechseln
cd Pfad/zu/Treydview

# gelieferte Dateien an die richtigen Orte kopieren:
# app.js -> js/app.js
# style.css -> css/style.css
# index.html -> index.html (Wurzelordner)

git add -A
git commit -m "m84: kurze Beschreibung der Änderung"
git push origin main
```

Die drei Git-Zeilen bedeuten im Klartext:

Befehl	Was er tut
<code>git add -A</code>	Merkt alle geänderten Dateien zum Hochladen vor.
<code>git commit -m "..."</code>	Hält daraus einen benannten Schnappschuss fest – vorerst nur lokal auf deinem Rechner. Der Text in Anführungszeichen ist die Notiz dazu (sinnvoll: die Build-Nummer und ein Stichwort).
<code>git push origin main</code>	Lädt den Schnappschuss zu GitHub hoch. Erst dieser Schritt macht die Änderung öffentlich.

Was nach dem Push von selbst passiert

GitHub startet automatisch die Veröffentlichung. Den Fortschritt siehst du auf `github.com/panta-rey/Treydview` im Reiter **«Actions»**: ein gelber Punkt bedeutet «läuft gerade», ein grüner Haken «fertig» (nach etwa 1–2 Minuten). Danach ist die neue Version live.

Danach die Seite mit einem **harten Reload** prüfen: `Ctrl + F5` (Windows) bzw. `Cmd + Shift + R` (Mac). Dann F12 öffnen und schauen, ob die grüne Build-Meldung erscheint und keine roten Fehler stehen.

Notfall-Ablauf (Historie divergiert, Push abgelehnt)

Wenn `git push` mit **«rejected»** oder **«non-fast-forward»** abbricht, ist die lokale Historie von der auf GitHub abgewichen. Der bewährte Rettungsweg:

```
# 1) gelieferte Dateien ZUERST an einem sicheren Ort ablegen (Kopie ausserhalb des Repos)

git rebase --abort          # nur nötig, falls ein Rebase feststeckt
git reset --hard origin/main # lokalen Stand exakt auf GitHub zurücksetzen

# 2) jetzt die gelieferten Dateien erneut an die richtigen Pfade kopieren
# (js/app.js, css/style.css, index.html)

git add -A
git commit -m "m84: Beschreibung"
git push origin main
```

Vorsicht mit `git reset --hard`

Dieser Befehl **verwirft alle lokalen, noch nicht gepushten Änderungen unwiderruflich**. Deshalb die gelieferten Dateien immer erst ausserhalb des Repos sichern, dann zurücksetzen, dann zurückkopieren.

07 Die Pflichtprüfungen vor jedem Push

Diese Checks fangen Fehler ab, **bevor** sie live gehen. In einer Arbeitssitzung mit dem KI-Assistenten laufen sie automatisch; wer selbst Hand anlegt, führt sie so aus. (Node.js muss installiert sein.)

Prüfung	Befehl	Fängt...
1. JS-Syntax je geänderte Datei	<code>node -c js/app.js</code>	Klammer-/Tippfehler. Kein Output = alles gut.
2. undefinierte Namen ESLint <code>no-undef</code>	<code>npx eslint js/app.js</code>	vergessene Deklarationen, die sonst still in einem <code>try/catch</code> verschwinden und ein ganzes Feature löschen können.

3. CSS-Klammern	<pre>grep -o '{' css/style.css wc -l grep -o '}' css/style.css wc -l</pre>	fehlende geschweifte Klammern. Beide Zahlen müssen gleich sein.
4. VM-Ladetest	<pre>node /tmp/loadtest.js</pre>	Startfehler (ReferenceError / temporale Todeszone), die erst beim Laden auftauchen.

Warum ausgerechnet diese vier?

Jede davon hat schon einmal einen echten Ausfall verhindert oder verursacht: Ein vergessenes `labelMode` wurde von einem `try/catch` verschluckt und löschte zwei Builds lang alle Projektionen. Eine fehlende `retryTimer`-Deklaration killte still das Laden eines Layouts. Genau solche Fehler machen keinen sichtbaren Absturz – das Feature ist einfach weg. Deshalb ESLint `no-undef` und der VM-Ladetest.

Zusätzlich («Prüfung 1»): Beim Ansehen des `style.css`-Unterschieds (Diff) dürfen **nur beabsichtigte Änderungen** auftauchen. Steht dort mehr, wurde versehentlich am Desktop-Aussehen gerüttelt.

08 Der Cloudflare-Worker

Der gesamte Worker-Code steht in einer einzigen Datei: `worker-komplett.js`. Er beantwortet diese Anfragen («Routen»):

Route	Wofür
/	Status-Check – zeigt JSON mit allen Routen. Gut zum Testen, ob der Worker überhaupt lebt.
/macro	FRED-Makrodaten + Gold
/goldhistory	Gold XAU/USD ab 1968 (LBMA)
/silverhistory	Silber XAG/USD ab 1968 (LBMA)
/bitstamp	BTC/ETH USD-Historie ab 2011
/stoq	Aktienindizes & Fonds (S&P 500, Nasdaq, Dow, QQQ, VTSAX)
/m2	globale M2-Geldmenge (FRED, aggregiert)
/dominance	BTC.D / USDT.D (CoinGecko Top-40)
/history · /snapshot	Snapshots lesen / schreiben
/sync	Einstellungen synchronisieren (Sync-Code, SHA-256-gehasht)

Wie man den Worker aktualisiert

Der Worker liegt **nicht** auf GitHub, sondern bei Cloudflare. Zum Aktualisieren:

- Bei Cloudflare einloggen → **Workers & Pages** → den Worker wählen.
- Edit code** (Quick Edit) öffnen, den kompletten Inhalt von `worker-komplett.js` hineinkopieren.
- Deploy / Save and deploy** klicken. Sofort aktiv.

Alternativ per Kommandozeile mit Cloudflare Wrangler (`wrangler deploy`), falls eingerichtet.

Bindings, die gesetzt sein müssen

Ohne diese Einstellungen im Cloudflare-Dashboard funktionieren einzelne Routen nicht:

Binding	Typ	Wofür
PANTA	KV-Namespace	Cache & Speicher für Sync-Codes. Pflicht.
FRED_KEY	Secret	FRED-API-Schlüssel (Makro-Daten).
FRED_API_KEY	Secret	optional, für die M2-Route – gleicher Wert wie FRED_KEY.
SNAP_KEY	Secret	optional, schützt die <code>/snapshot</code> -Route vor Fremdzugriff.
COINGECKO_KEY	Secret	optional, für <code>/dominance</code> (höheres Limit).

Der Cache-Fallstrick: `CACHE_VERSION`

Ganz oben in `worker-komplett.js` steht `const CACHE_VERSION = "v2";`. Jeder Cache-Schlüssel trägt diese Kennung. **Wenn du am Rechenweg einer Datenquelle etwas änderst, musst du diese Kennung erhöhen** (v2 → v3). Sonst gibt der Worker bis zu 24 Stunden lang weiter die **alten, falschen Werte** aus dem Cache aus – und dein Fix sieht aus, als hätte er nicht gewirkt. Genau das ist bei den M2-Werten einmal passiert.

09 Datenquellen & ihre Eigenheiten

Nicht jede Quelle ist gleich zuverlässig. Diese Übersicht hilft beim Einordnen, wenn etwas nicht lädt.

Quelle	Weg	Was & Eigenheiten
Binance	direkt (WS + REST)	Krypto Live-Preise + Historie ab Aug 2017. Zuverlässig. Einzige Quelle mit Live-Stream.

Coinbase	direkt (REST)	nur bis Tageschart (kein Weekly/Monthly), kein Live-Stream .
Kraken	direkt (REST)	kein Live-Stream (Anzeige zeigt «Kraken» statt «Live»). Historie teils bis 2013.
Bybit	direkt (REST)	kein Live-Stream.
Bitstamp	über Worker	BTC/ETH USD ab 2011. Liefert 1h/4h/Tag; Wochen/Monate werden im Browser aggregiert.
LBMA	über Worker	Gold & Silber ab 1968. Fixing-Preise (kein echtes OHLC, kein Volumen).
CoinGecko	über Worker	Dominanz (BTC.D/USDT.D), Top-40-Summe, 24h-Cache.
Stooq / FRED	über Worker	Indizes & M2. Direktabruf unmöglich – Stooq liefert JS-Challenge-Seiten, Yahoo antwortet mit 429. Nur über den Worker mit Cache brauchbar.

Zwei bekannte Rechenfallen (aus der Projekthistorie)

- Die FRED-Serie für ausländische Geldmengen (IWF) meldet **Landeswährung, nicht Milliarden** – beim Aggregieren beachten.
- Wochengrenzen: Für Montag 00:00 UTC muss von der Donnerstag-Unix-Epoche **–4 Tage** gerechnet werden (nicht +4). Ein falsches Vorzeichen verschob den 200-Wochen-SMA um rund 25 Punkte.

10 Statische Historie (Snapshots)

Die drei Dateien in `data/` enthalten die komplette Altdaten-Historie (BTC, ETH, Gold). Der Browser lädt sie vom GitHub-CDN und fragt den Worker danach nur noch nach dem **Zuwachs seit der letzten gespeicherten Kerze**. Das ist schneller (ca. 62 KB für BTC, 109 KB für Gold, gzip) und funktioniert sogar, wenn Worker oder Quelle gerade nicht erreichbar sind.

Robust gebaut

Fehlt eine Snapshot-Datei, fällt der Ladeweg automatisch auf den vollen Worker-Abruf zurück – nichts geht kaputt, es wird nur langsamer. Man kann eine Datei also gefahrlos entfernen.

Snapshots aktualisieren

Nur bei Bedarf (die Dateien wachsen ohnehin über den Zuwachs-Mechanismus mit). Alles Folgende läuft im **Terminal** (Kapitel 4), im Projektordner, und setzt **Python 3** voraus (siehe Voraussetzungen in Kapitel 4). Zuerst das Skript ausführen, das die JSON-Dateien neu schreibt:

```
bash snapshot.sh
```

Die erste Zeile der Ausgabe zeigt, welches Python verwendet wird (z. B. `Python: python (Python 3.13...)`). Fehlt ein funktionierendes Python, bricht das Skript sofort mit einer klaren Meldung ab, statt halb fertige Dateien zu hinterlassen.

`bash` startet das Skript `snapshot.sh`; es holt die aktuellen Tageskerzen und legt die Dateien in `data/` neu an. Danach werden diese Dateien – wie jede andere Änderung – zu GitHub hochgeladen:

```
git add data/ && git commit -m "Snapshots aktualisiert" && git push origin main
```

Das sind dieselben drei Schritte wie beim normalen Deploy (Kapitel 6), hier nur auf den Ordner `data/` beschränkt. Die Zeichen `&&` hängen die Befehle aneinander: Sie laufen nacheinander und nur dann weiter, wenn der vorige geklappt hat. Man könnte die drei auch einzeln auf drei Zeilen eingeben – das Ergebnis ist dasselbe.

Hier keine Build-Nummer erhöhen

Es ändert sich nur der Ordner `data/`, kein JavaScript und kein CSS. Die Build-Nummer (Kapitel 5) bleibt also unangetastet – sie ist nur für Änderungen an `app.js`, `style.css` oder `index.html` nötig.

Nur Tageskerzen – für 1h/4h wären es Hunderttausende Kerzen, die gehören nicht ins Repo. Diese Intervalle laufen weiter über den vollen Abruf.

11 Troubleshooting

Format: **Symptom** (rot) → wahrscheinliche **Ursache** → **Behebung**. Der erste Griff bei fast jedem Problem: F12 drücken und die **Konsole** sowie den **Netzwerk**-Tab ansehen.

Nach dem Deploy sehe ich die Änderung nicht.

URSACHE Browser-Cache, oder Build-Nummer vergessen zu erhöhen.

BEHEBUNG Harter Reload (`Ctrl + F5`). Prüfen, ob alle drei Build-Stellen erhöht sind (Kapitel 5). In der F12-Konsole auf die grüne `Build-aktuell`-Meldung achten.

Konsole zeigt: [TreydView] VERSIONSKONFLIKT — JS "mXX", CSS "mYY"

URSACHE `TV_BUILD` (app.js) und `--tv-build` (style.css) stimmen nicht überein – eine Stelle wurde beim Bumpen vergessen.

BEHEBUNG Beide auf denselben Wert setzen, neu deployen, hart neu laden.

Der Chart bleibt auf «Lade ...» oder bleibt leer.

URSACHE Datenquelle/Worker nicht erreichbar, oder ein JavaScript-Fehler beim Start.

BEHEBUNG F12-Konsole öffnen. Ein roter `ReferenceError` bedeutet, dass eine Deklaration fehlt (ESLint `no-undef` hätte das gefangen). Im Netzwerk-Tab prüfen, ob die Worker-Aufrufe Status `200` liefern.

Gold/Silber/Indizes/Dominanz laden nicht - Krypto schon.

URSACHE Worker-Problem oder die Worker-URL wurde überschrieben (Regel 1!).

BEHEBUNG In `config.js` die `WORKER_BASE_URL` prüfen. Dann die Worker-Root-Route im Browser aufrufen (`...workers.dev/`) – sie sollte JSON mit einer `routes`-Liste zeigen. Tut sie das nicht, ist der Worker selbst das Problem (Cloudflare-Dashboard, Kapitel 8).

Falsche/veraltete Makro- oder M2-Werte, obwohl der Fix drin ist.

URSACHE Der KV-Cache im Worker liefert noch die alten Werte.

BEHEBUNG `CACHE_VERSION` im Worker erhöhen (v2 → v3) und den Worker neu deployen.

«Keine plausiblen M2-Daten erhalten»

URSACHE Alte, fehlerhafte M2-Werte (Faktor 1e9 zu gross) stecken im Cache und werden vom Frontend als unplausibel verworfen.

BEHEBUNG `CACHE_VERSION` erhöhen, Worker neu deployen.

Statusanzeige zeigt «Kraken» oder «Bybit» statt «Live».

URSACHE Diese Börsen haben keinen Live-Stream.

BEHEBUNG Kein Fehler – erwartetes Verhalten.

git push wird abgelehnt («rejected» / «non-fast-forward»).

URSACHE Die lokale Historie ist von GitHub abgewichen.

BEHEBUNG Notfall-Ablauf aus Kapitel 6: Dateien sichern → `git reset --hard origin/main` → Dateien zurückkopieren → commit → push.

Ein Feature/Indikator ist nach einer Änderung einfach verschwunden.

URSACHE Ein still verschluckter Fehler (z. B. `ReferenceError` in einem `try/catch`) – kein Absturz, aber das Feature läuft ins Leere.

BEHEBUNG ESLint `no-undef` laufen lassen und den VM-Ladetest. Beide zeigen den fehlenden Namen.

Dominanz zeigt btcd=100 / usdtd=null.

URSACHE Problem in der `/dominance`-Antwort (CoinGecko) – offener Punkt aus der Projekthistorie.

BEHEBUNG Die `/dominance`-Route direkt im Browser aufrufen und die Rohantwort ansehen. Ggf. `CACHE_VERSION` erhöhen, um den Cache zu leeren.

Ein Preis-Label an der Achse ist unleserlich (heller Text auf heller Linie).

URSACHE KLineCharts erlaubt nur eine globale Textfarbe für die Achsen-Labels; eine zu helle Linienfarbe wird unlesbar.

BEHEBUNG Jede neue Linienfarbe muss gegen den dunklen Text `#0d1117` mindestens Kontrast 4.5 haben (Helfer `contrastRatio` / `textForLines` in `config.js`).

Das Handy (als App vom Home-Screen) zeigt nach einem Update alte Inhalte.

URSACHE Die installierte PWA hält eine zwischengespeicherte Version.

BEHEBUNG Sicherstellen, dass `manifest.webmanifest?v=` mit-erhöht wurde. Im Zweifel die App vom Home-Screen entfernen und neu hinzufügen.

12 Schnell-Checkliste vor jeder Auslieferung

- ☐ Build-Nummer an **allen drei** Stellen erhöht: `TV_BUILD` (app.js), `--tv-build` (style.css), `?v=` (index.html, 14x)
- ☐ `node -c` auf allen geänderten JS-Dateien – kein Output
- ☐ ESLint `no-undef` sauber
- ☐ CSS – öffnende und schliessende geschweifte Klammern gleich viele
- ☐ VM-Ladetest grün
- ☐ style.css-Diff enthält **nur** beabsichtigte Änderungen (Desktop unangetastet)
- ☐ `WORKER_BASE_URL` in config.js unverändert
- ☐ `HANDOFF.md` ist **nicht** im Commit
- ☐ Dateien an den richtigen Pfaden: `js/`, `css/`, `index.html` im Wurzelordner
- ☐ `git push` → harter Reload → F12-Konsole: grüne Build-Meldung, keine roten Fehler